ภาพรวมของบริการ (Service Overview)

Disaster Scraping Service

1. **Service Owner**

   นางสาว ชัญญานุช อิ่มเกิด รหัสนักศึกษา 6609650277 ภาคพิเศษ

2. **Service Purpose**

   Disaster Scraping Service เป็นบริการที่ทำหน้าที่รวบรวมข้อมูลการแจ้งเตือนสภาพอากาศและภัยพิบัติจากเพจ Facebook กรมอุตุนิยมวิทยาโดยอัตโนมัติผ่านเทคนิค web scraping เพื่อใช้เป็นแหล่งข้อมูลตั้งต้นสำหรับการตรวจจับเหตุการณ์ภัยพิบัติในระบบตอบสนองภัยพิบัติ

   บริการนี้ช่วยแปลงข้อมูลจากหน้าเว็บที่ไม่มี API ให้เป็นข้อมูลเชิงโครงสร้าง (structured data) และเผยแพร่เป็น event หรือ API ให้บริการอื่น เช่น Incident Service สามารถนำไปใช้งานต่อได้

3. **Pain Point ที่แก้ไข**

   ในปัจจุบัน การแจ้งเตือนภัยพิบัติมักอยู่ในรูปแบบ

   - หน้าเว็บประกาศ
   - ข่าวเตือนภัย
   - ข้อความที่ไม่มี API

   ทำให้ระบบอัตโนมัติไม่สามารถนำข้อมูลไปใช้งานได้โดยตรง และต้องอาศัยการติดตามด้วยมนุษย์ ซึ่งมีความล่าช้า

   บริการนี้จึงช่วย

   - ดึงข้อมูลประกาศเตือนภัยแบบอัตโนมัติ
   - ตรวจจับเหตุการณ์ใหม่ได้เร็วขึ้น
   - ลด Manual monitoring
   - ทำให้ระบบ disaster response เป็น real-time มากขึ้น

3. Target Users

- Incident Management Service
- Dashboard แจ้งเตือนภัยพิบัติ

บริการนี้เป็น backend data ingestion service ไม่ได้ให้ผู้ใช้งานทั่วไปเรียกโดยตรง

4. Service Boundary

In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)

- ทำ web scraping จาก Facebook กรมอุตุนิยมวิทยา
- ตรวจจับประกาศเตือนภัยใหม่
- แปลงข้อมูลจาก HTML เป็น structured data
- กรอง duplicate announcements
- ส่งข้อมูลออกเป็น event หรือ API

Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)

- การจัดระดับความรุนแรงของภัยพิบัติ

- การจัดการศูนย์พักพิง
- การวิเคราะห์ผลกระทบภัยพิบัติ
- การแจ้งเตือนผู้ใช้งาน

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ

- การตรวจจับว่า announcement ใดเป็นข้อมูลใหม่
- การ normalize ข้อมูลวันที่และเวลา
- การตัดสินใจ publish event

ตรรกะการตัดสินใจอิงจาก

- announcement title
- timestamp
- content keyword

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอ/ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

1. Scraped Announcement Data

ข้อมูลประกาศที่ดึงมาจาก Facebook Page หรือแหล่งต้นทางในรูปแบบที่ยังใกล้เคียงต้นฉบับ เช่น ชื่อโพสต์ เนื้อหา วันที่โพสต์ ลิงก์ เป็นต้น ข้อมูลส่วนนี้ถูกเก็บไว้เพื่อใช้ในการตรวจสอบย้อนหลัง (audit) และเพื่อให้สามารถ re-process ได้ในกรณีที่ต้องปรับปรุง logic การวิเคราะห์ในอนาคต

2. Normalized Disaster Event Draft

ข้อมูลที่ผ่านกระบวนการแปลง (normalization) จากข้อความดิบให้เป็น structured format เช่น eventType, severity, publishedAt และ affectedArea ข้อมูลส่วนนี้เป็นผลลัพธ์จาก business rules ภายใน service และเป็นข้อมูลที่พร้อมส่งต่อให้ Incident Service หรือระบบอื่นใช้งาน

3. Scraping Metadata

ข้อมูลประกอบกระบวนการ scraping เช่น เวลาในการดึงข้อมูล (scrapedAt), แหล่งที่มา (sourceUrl), response status และ content hash เพื่อใช้ในการติดตามประสิทธิภาพของระบบ ตรวจสอบปัญหา (debugging) และรองรับการตรวจสอบย้อนหลัง

4. Deduplication State

ข้อมูลที่ใช้ป้องกันการส่ง event ซ้ำ เช่น hash ของเนื้อหา, postId หรือ eventId โดยจะถูกใช้ตรวจสอบก่อน publish event ทุกครั้ง เพื่อรองรับรูปแบบการส่งข้อมูลแบบ at-least-once และป้องกันการสร้าง incident ซ้ำในระบบ downstream

7. Linked Data (Reference Only)

ระบบจะตรวจสอบ incident จาก Incident Service โดย Incident Service ได้ข้อมูลจาก

Web scraping Facebook กรมอุตุนิยมวิทยา เช่น ประกาศเตือนพายุ ประกาศฝนตกหนัก แจ้งเตือนน้ำท่วม

8. Non-Functional Requirements

- ใช้ at-least-once event delivery
- ต้องมี idempotency ในการ publish event
- Retry scraping หาก request ล้มเหลว (ไม่เกิน 3 ครั้ง)
- ระบบต้องไม่ crash เมื่อ HTML เปลี่ยนรูปแบบ
- ต้องมี timeout และ error handling
- ต้องเคารพ rate limit ของเว็บไซต์ต้นทาง

API Contracts

Synchronous APIs (REST)

1. **Trigger Scraping Job**

General Information

- Name: Trigger Scraping Job
- Method: POST
- Path: /v1/scraping-jobs
- Description: สั่งให้ระบบเริ่มกระบวนการ scraping ทันที

Request

Headers:

Content-Type: application/json

Body:

```
{
  "source": "FACEBOOK_PAGE"
}
```

Validation Rules

- source ต้องเป็น enum ที่กำหนด
- ห้ามมี job ที่สถานะ RUNNING อยู่
- ต้องมี Authorization header

Response

202 Accepted

```
{
  "traceId": "abc123",
  "jobId": "job-001",
  "status": "RUNNING"
}
```

ความหมาย: ระบบรับคำสั่งและเริ่ม job แล้ว

Error

400 Invalid input

401 Unauthorized

409 Job already running

Dependency / Reliability

- ไม่เรียก service อื่น
- Timeout: 5 วินาที
- Retry: 3 ครั้งภายใน worker
- Idempotent: ไม่ใช่
- หาก scraping engine ล่ม ทำการ log และ retry

2. **Get Scraped Posts**

General Information

- Name: Get Scraped Posts
- Method: GET
- Path: /v1/posts
- Description: ดึงโพสต์ที่ scrape มาแล้วในรูปแบบ raw text

Query Parameters

Name	Type	Required	Default	Description
fromDate	string	optional	null	กรองตั้งแต่วันที่
toDate	string	optional	null	กรองถึงวันที่
limit	int	optional	20	จำนวนสูงสุด

Validation Rules

- limit ≤ 100
- fromDate ≤ toDate
- วันที่ต้องเป็น ISO8601

Response

200 OK


```
{
  "traceId": "abc123",
  "data": [
    {
      "postId": "fb-001",
      "messageText": "ประกาศเตือนฝนตกหนัก...",
      "publishedAt": "2026-03-03T10:30:00Z",
      "sourceUrl": "https://facebook.com/...",
      "scrapedAt": "2026-03-03T10:35:00Z"
    }
  ]
}
```


Idempotent: ใช่

Reliability

- Read-only
- Timeout: 5 วินาที
- ไม่ retry

3. **Get Post By ID**

General Information

- Method: GET
- Path: /v1/posts/{postId}
- Description: ดึงโพสต์เฉพาะรายการตาม postId

Path Parameter

Name	Type	Required	Description
postId	string	yes	รหัสโพสต์

Response

200 OK

```
{
  "traceId": "abc123",
  "postId": "fb-001",
  "messageText": "...",
  "publishedAt": "...",
  "sourceUrl": "...",
  "scrapedAt": "..."
}
```

404 Not Found

```
{
  "traceId": "abc123",
  "message": "Post not found"
}
```

Idempotent: ใช่

4. **Get Scraping Job Status**

General Information

- Method: GET
- Path: /v1/scraping-jobs/{jobId}
- Description: ตรวจสอบสถานะของ scraping job

Path Parameter

Name	Type	Required

jobId	string	yes

Response

```
{
  "traceId": "abc123",
  "jobId": "job-001",
  "status": "COMPLETED",
  "startedAt": "...",
  "completedAt": "..."
}
```

Idempotent: ใช่

Asynchronous Function

5. **DisasterPostScraped Event**

General Information

- Message Name: DisasterPostScraped
- Interaction Style: Publish–Subscribe
- Role: Producer

Description: ส่งข้อความเมื่อพบโพสต์ใหม่ที่ยังไม่เคย publish มาก่อน

Incident Service จะ subscribe channel นี้เพื่อนำข้อความไปวิเคราะห์เอง

Message Contract

Header

```
{
  "messageId": "msg-123",
  "timestamp": "2026-03-03T10:31:00Z",
  "version": "v1",
  "source": "scraping-service"
}
```

Body

```
{
  "postId": "fb-001",
  "messageText": "ประกาศเตือนฝนตกหนัก...",
  "publishedAt": "2026-03-03T10:30:00Z",
  "sourceUrl": "https://facebook.com/...",
  "contentHash": "abcxyz123"
}
```

Field Definition (Body)

Field	Type	Required	Description

postId	string	yes	รหัสโพสต์
messageText	string	yes	ข้อความดิบ
publishedAt	string	yes	วันที่โพสต์
sourceUrl	string	yes	ลิงก์ต้นทาง
contentHash	string	yes	ใช้ป้องกัน Duplicate

Response (Acknowledgment Model)

Success

Header:

```
{
  "messageId": "msg-123",
  "status": "ACK"
}
```

Body:

```
{
  "processed": true
}
```

Reject

Header:

```
{
  "messageId": "msg-123",
  "status": "REJECT"
}
```

Body:

```
{
  "errorCode": "DUPLICATE_POST",
  "message": "Post already processed"
}
```

Validation Rules

- postId ต้อง unique
- messageText ต้องไม่ว่าง
- publishedAt ต้องไม่เป็น future date
- Reliability Design
- Delivery: at-least-once
- Retry: 3 ครั้ง (exponential backoff)
- Consumer ต้องรองรับ idempotency
- หาก downstream ล่ม message จะค้างใน queue
- Timeout consumer: 10 วินาที

Service Data

Disaster Scraping Service เป็นเจ้าของข้อมูลที่เกี่ยวข้องกับกระบวนการ scraping และการเผยแพร่ข้อความประกาศจาก Facebook Page โดยไม่ทำหน้าที่วิเคราะห์หรือจัดประเภทภัยพิบัติ ข้อมูลหลักที่จัดเก็บภายใน service แบ่งออกเป็น 3 กลุ่มสำคัญ ได้แก่ ข้อมูลโพสต์ที่ถูก scrape, ข้อมูลสถานะของ scraping job และข้อมูลที่ใช้สำหรับป้องกันการส่งข้อความซ้ำ

ข้อมูลประเภทแรกคือ **ScrapedPost** ซึ่งเก็บข้อมูลโพสต์ในรูปแบบข้อความดิบ (raw text) ตามต้นฉบับ โดยประกอบด้วย postId, messageText, publishedAt, sourceUrl และ scrapedAt ข้อมูลชุดนี้ถือเป็น internal source of truth ภายในขอบเขตของ service และรองรับการใช้งานผ่าน API แบบ synchronous เช่น GET /v1/posts และ GET /v1/posts/{postId} รวมถึงรองรับการตรวจสอบย้อนหลัง (audit) และการ reprocess หากจำเป็น

ข้อมูลประเภทที่สองคือ **ScrapingJob** ซึ่งใช้ติดตามสถานะของกระบวนการ scraping เช่น jobId, status (RUNNING, COMPLETED, FAILED), startedAt และ completedAt ข้อมูลนี้สอดคล้องกับ API เช่น POST /v1/scraping-jobs และ GET /v1/scraping-jobs/{jobId} เพื่อให้สามารถตรวจสอบความคืบหน้าและควบคุมการทำงานแบบ concurrent ได้

ข้อมูลประเภทที่สามคือ **DeduplicationState** ซึ่งใช้ contentHash หรือ postId เป็น unique key เพื่อป้องกันการ publish event ซ้ำ เนื่องจากระบบออกแบบให้รองรับ at-least-once delivery ในการสื่อสารแบบ asynchronous การมี state นี้ช่วยให้ service สามารถตรวจสอบก่อนส่ง DisasterPostScraped event และลดความเสี่ยงในการสร้างข้อมูลซ้ำในระบบ downstream

ในส่วนของ Linked Data บริการนี้อ้างอิงเพียง Facebook Page ในฐานะแหล่งข้อมูลต้นทาง และไม่จัดเก็บหรือสร้าง incident entity ภายใน service เอง เพื่อหลีกเลี่ยงความซ้ำซ้อนกับ Incident Service และรักษาขอบเขตความรับผิดชอบตามหลัก microservices อย่างชัดเจน

การออกแบบข้อมูลลักษณะนี้ทำให้ service สามารถรองรับทั้งการเรียกใช้งานแบบ synchronous (query และตรวจสอบสถานะ) และ asynchronous (publish raw post event) ได้อย่างสอดคล้องกัน ข้อมูลที่จัดเก็บรองรับการทำงานพร้อมกัน (concurrent execution), การตรวจสอบย้อนหลัง, การป้องกัน duplicate และการทำงานในสภาพแวดล้อมแบบ distributed systems ได้อย่างเหมาะสม

โดยจะเก็บข้อมูลเป็นตาราง ดังนี้

1. **ScrapedPost**

เก็บข้อมูลโพสต์ที่ scrape มาในรูปแบบข้อความดิบ (raw text) เป็น internal source of truth ของ service

Field	Type	Required	Description
postId	string	yes	รหัสโพสต์จาก Facebook

messageText	text	yes	ข้อความดิบจากโพสต์
publisedAt	datetime	yes	วันที่โพสต์
sourceUrl	string	yes	ลิงก์ต้นทาง
scrapedAt	datetime	yes	เวลาที่ scrape
contentHash	string	yes	hash ของข้อความ ใช้ป้องกันซ้ำ

2. **ScrapingJob**

ใช้ติดตามสถานะของการทำ scraping รองรับ API /v1/scraping-jobs และ /v1/scraping-jobs/{jobId}

Field	Type	Required	Description
jobId	string	yes	รหัส job
source	string	yes	แหล่งข้อมูล
status	enum	yes	สถานะ job RUNNING / COMPLETED / FAILED
startedAt	datetime	yes	เวลาเริ่ม
completedAt	datetime	no	เวลาจบ
totalPostsFetched	int	no	จำนวนโพสต์ที่ดึงได้
errorMessage	string	no	ข้อความ error ถ้ามี

3. **DeduplicationState**

ใช้ป้องกันการ publish event ซ้ำ รองรับ at-least-once delivery

Field	Type	Required	Description
contentHash	string	yes	hash ของข้อความ
postId	string	yes	อ้างถึง ScrapedPost
publishedToQueue	boolean	yes	เคย publish แล้วหรือไม่
createdAt	datetime	yes	เวลาเก็บ record

Link GitHub: https://github.com/ChanyanuchImk/cs366_Microservice_0277.git